

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

数值

- **Infinity**表示“无穷”，用来表示两种场景。一种是一个正的数值太大，或一个负的数值太小，无法表示；另一种是非0数值除以0，得到Infinity

```
// 场景一
Math.pow(2, 1024)
// Infinity

// 场景二
0 / 0 // NaN
1 / 0 // Infinity
```

数值

- Infinity表示正无穷，-Infinity表示负无穷

```
Infinity === -Infinity // false
```

```
1 / -0 // -Infinity
```

```
-1 / -0 // Infinity
```

- Infinity大于一切数值（除了NaN），-Infinity小于一切数值（除了NaN），Infinity与NaN比较，总是返回false

```
Infinity > NaN // false
```

```
-Infinity > NaN // false
```

```
Infinity < NaN // false
```

```
-Infinity < NaN // false
```

数值

- Infinity的运算

- Infinity的四则运算，符合无穷的数学计算规则
- 0乘以Infinity，返回NaN；0除以Infinity，返回0；Infinity除以0，返回Infinity
- Infinity加上或乘以Infinity，返回的还是Infinity
- Infinity减去或除以Infinity，得到NaN
- Infinity与null计算时，null会转成0，等同于与0的计算
- Infinity与undefined计算，返回的都是NaN

与数值相关的全局方法

- **parseInt**方法用于将字符串转为整数，parseInt方法还可以接受第二个参数（2到36之间），表示被解析的值的进制，返回该值对应的十进制数。默认是转十进制。如果字符串的第一个字符不能转化为数字（后面跟着数字的正负号除外），返回NaN

```
parseInt('123') // 123
```

```
> parseInt(123,20)  
◁ 443
```

```
parseInt(' 81') // 81
```

如果字符串头部有空格，空格会被自动去除

```
parseInt(1.23) // 1  
// 等同于  
parseInt('1.23') // 1
```

如果parseInt的参数不是字符串，则会先转为字符串再转换

与数值相关的全局方法

- parseInt方法用于将字符串转为整数

```
parseInt('8a') // 8  
parseInt('12**') // 12  
parseInt('12.34') // 12  
parseInt('15e2') // 15  
parseInt('15px') // 15
```

字符串转为整数的时候，是一个个字符依次转换，如果遇到不能转为数字的字符，就不再进行下去，返回已经转好的部分

```
parseInt('abc') // NaN  
parseInt('.3') // NaN  
parseInt('') // NaN  
parseInt('+') // NaN  
parseInt('+1') // 1
```

转换出错时返回NaN

```
parseInt('0x10') // 16
```

如果字符串以0x或0X开头，parseInt会将其按照十六进制数解析

与数值相关的全局方法

- **parseFloat**方法用于将一个字符串转为浮点数

```
parseFloat('3.14') // 3.14
```

```
parseFloat('314e-2') // 3.14  
parseFloat('0.0314E+2') // 3.14
```

```
parseFloat('3.14more non-digit characters') // 3.14
```

与数值相关的全局方法

- **isNaN** 方法可以用来判断一个值是否为NaN，isNaN只对数值有效，如果传入其他值，会被先转成数值

```
isNaN(NaN) // true  
isNaN(123) // false
```

```
isNaN('Hello') // true  
// 相当于  
isNaN(Number('Hello')) // true
```

比如，传入字符串的时候，字符串会被先转成NaN，所以最后返回true，同理对于对象和数组，isNaN也返回true

Number是JavaScript的内置函数，可以将给定的参数转换为数字类型。当给定的参数无法转换为数字时，Number函数会返回NaN（非数字）。对于布尔值true和false，它们会被转换成1和0，而对于空字符串""，它会被转换成0。

与数值相关的全局方法

- 对于空数组和只有一个数值成员的数组，`isNaN`返回`false`，原因是这些数组能被`Number`函数转成数值，使用`isNaN`之前，最好判断一下数据类型
- 判断NaN更可靠的方法是，利用NaN是唯一一个不等于自身的值的这个特点，进行判断

```
function myIsNaN(value) {  
  return value !== value;  
}
```

```
>> Number([])
```

```
← 0
```

```
>> Number([100])
```

```
← 100
```

```
> Number([100, 20, 30])
```

```
← NaN
```

与数值相关的全局方法

- **isFinite**方法返回一个布尔值，表示某个值是否为正常的数值

```
isFinite(Infinity) // false
isFinite(-Infinity) // false
isFinite(NaN) // false
isFinite(undefined) // false
isFinite(null) // true
isFinite(-1) // true
```

除了Infinity、-Infinity、NaN和undefined这几个值会返回false，isFinite对于其他的数值都会返回true

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

字符串

- 字符串就是零个或多个排在一起的字符，放在单引号或双引号之中

```
var longString = 'Long \
long \
long \
string';
```

字符串默认只能写在一行内，分成多行将会报错
如果长字符串必须分成多行，可以在每一行的尾部使用反斜杠

```
longString  
// "Long long long string"
```

```
var longString = 'Long '  
+ 'long '  
+ 'long '  
+ 'string';
```

连接运算符 (+) 可以连接多个单行字符串，将长字符串拆成多行书写，输出的时候也是单行

字符串

- 反斜杠 (\) 在字符串内有特殊含义，用来表示一些特殊字符，所以又称为转义符

- `\0` : null (`\u0000`)
- `\b` : 后退键 (`\u0008`)
- `\f` : 换页符 (`\u000C`)
- `\n` : 换行符 (`\u000A`)
- `\r` : 回车键 (`\u000D`)
- `\t` : 制表符 (`\u0009`)
- `\v` : 垂直制表符 (`\u000B`)
- `\'` : 单引号 (`\u0027`)
- `\"` : 双引号 (`\u0022`)
- `\\` : 反斜杠 (`\u005C`)

字符串

- 字符串可以被视为字符数组，因此可以使用数组的方括号运算符，用来返回某个位置的字符（位置编号从0开始）

```
var s = 'hello';
```

```
s[0] // "h"
```

```
s[1] // "e"
```

```
s[4] // "o"
```

```
// 直接对字符串使用方括号运算符
```

```
'hello'[1] // "e"
```

这个字符串的长度是 6

位置	H	E	L	L	O	!
	0	1	2	3	4	5

```
'abc'[3] // undefined
```

```
'abc'[-1] // undefined
```

```
'abc'['x'] // undefined
```

如果方括号中的数字超过字符串的长度，或者方括号中根本不是数字，则返回undefined

字符串

- 字符串可以被视为字符数组，因此可以使用数组的方括号运算符，用来返回某个位置的字符（位置编号从0开始）

```
var s = 'hello';
```

```
delete s[0];
```

```
s // "hello"
```

```
s[1] = 'a';
```

```
s // "hello"
```

```
s[5] = '!';
```

```
s // "hello"
```

字符串内部的单个字符无法改变和增删，修改并没有生效，这些操作会默默地失败，不会返回错误信息



当心坑洞

字符串

- **length**属性返回字符串的长度，该属性也是无法改变的

```
var s = 'hello';
```

```
s.length // 5
```

```
s.length = 3;
```

```
s.length // 5
```

```
s.length = 7;
```

```
s.length // 5
```

字符串的length属性无法改变，
但是不会报错



当心坑洞

字符串

- JavaScript 不仅以 Unicode 储存字符，还允许直接在程序中使用 Unicode 码点表示字符，即将字符写成 `\uxxxx` 的形式，其中 `xxxx` 代表该字符的 Unicode 码点
- 解析代码的时候，JavaScript 会自动识别一个字符是 **字面形式表示**，还是 **Unicode 形式表示**。输出给用户的时候，所有字符都会转成字面形式

```
> var s = '\u11A9';  
s  
< "㐀"  
  
> var s = '\u01A9';  
s  
< "Σ"
```

```
var f\u006F\u006F = 'abc';  
foo // "abc"
```

第一行的变量名 `foo` 是 Unicode 形式表示，第二行是字面形式表示，JavaScript 会自动识别

字符串

- JavaScript 原生提供两个 Base64 相关的方法

- `btoa()` : 任意值转为 Base64 编码
- `atob()` : Base64 编码转为原来的值

```
var string = 'Hello World!';  
btoa(string) // "SGVsbG8gV29ybGQh"  
atob('SGVsbG8gV29ybGQh') // "Hello World!"
```

Base64 就是一种编码方法，可以将任意值转成 0~9、A~Z、a-z、+和/这64个字符组成的可打印字符

Base64编码用64个可打印字符表示二进制所有数据。由于2的6次方等于64，所以可以用每6个bit一个单元，对应某个可打印字符。**三个字节有24个位bit，就可以刚好对应于4个Base64单元，即3个字节需要用4个Base64的可打印字符来表示，数据不足3byte的话，缓冲区中剩下的bit用0补足，不足3个字节用=补足**

<http://www.cnblogs.com/chengmo/archive/2014/05/18/3735917.html>

字符串

- 这两个方法不适合非 ASCII 码的字符，会报错

```
> btoa('你好')
```

```
✖ ▶ Uncaught DOMException: Failed to execute 'btoa' on 'Window': The string to be encoded contains characters outside of the Latin1 range.  
    at <anonymous>:1:1
```

- 要将非 ASCII 码字符转为 Base64 编码（UTF-8 + 百分号编码），必须中间插入一个转码环节，再使用这两个方法

```
function b64Encode(str) {  
    return btoa(encodeURIComponent(str));  
}
```

```
function b64Decode(str) {  
    return decodeURIComponent(atob(str));  
}
```

```
b64Encode('你好') // "JUu0JUJEJUeWJUu1JUe1JUJE"  
b64Decode('JUu0JUJEJUeWJUu1JUe1JUJE') // "你好"
```

3 encodeURIComponent() 函数

定义和用法

encodeURIComponent() 函数可把字符串作为 URI 组件进行编码。

语法

encodeURIComponent(URIstring)

参数 描述

URIstring 必需。一个字符串，含有 URI 组件或其他要编码的文本。

返回值

URIstring 的副本，其中的某些字符将被十六进制的转义序列进行替换。

说明

该方法不会对 ASCII 字母和数字进行编码，也不会对这些 ASCII 标点符号进行编码：- _ . ! ~ * ' () 。

其他字符（比如：: ; ? @ & = + \$, # 这些用于分隔 URI 组件的标点符号），都是由一个或多个十六进制的转义序列替换的。

```
> encodeURIComponent("您好")  
< "%E6%82%A8%E5%A5%BD"  
> btoa("%E6%82%A8%E5%A5%BD")  
< "JUu0JTgyJUe4JUu1JUe1JUJE"
```

主要内容

- 基本语法
- 数据类型
 - null和undefined
 - 布尔值
 - 数值
 - 字符串
 - 对象
- 运算符
- 标准库

对象

- 对象（object）是 JavaScript 语言的核心概念，也是最重要的数据类型
- 对象就是一组“键值对”（key-value）的集合，是一种**无序**的复合数据集合

```
var obj = {  
  foo: 'Hello',  
  bar: 'World'  
};
```

对象的生成方法

大括号定义了一个对象，它被赋值给变量obj，所以变量obj就指向一个对象。该对象内部包含两个键值对（又称为两个“成员”），第一个键值对是foo: 'Hello'，其中foo是“键名”（成员的名称），字符串Hello是“键值”（成员的值）。键名与键值之间用冒号分隔。第二个键值对是bar: 'World'，bar是键名，World是键值。两个键值对之间用逗号分隔。

对象

- 对象的所有键名都是字符串，加不加引号都可以

```
var obj = {  
  'foo': 'Hello',  
  'bar': 'World'  
};
```



等同的

```
var obj = {  
  foo: 'Hello',  
  bar: 'World'  
};
```

对象

- 如果键名是数值，会被自动转为字符串

```
> var obj3 = {  
  1: 'e',  
  .31: 'e',  
  1.268: 'e',  
  1e-2: 'e',  
  0xFF: 'e'  
};
```

```
> obj3
```

```
◀ ▶ {1: 'e', 255: 'e', 0.31: 'e', 1.268: 'e', 0.01: 'e'}
```

对象obj的所有键名虽然看上去像数值，实际上都被自动当作 **number** 再转成 **string**

对象

- 如果键名不符合标识名的条件（比如第一个字符为数字，或者含有空格或运算符），且也不是数字，则必须加上引号，否则会报错

```
>> var obj = {  
    '1p': 'Hello 1p',  
    'h w': 'Hello h w',  
    'p+q': 'Hello p+q',  
    '***': 'Hello ***',  
    '-ddd': 'Hello -ddd'  
};  
← undefined  
>> obj  
← ► Object { 1p: "Hello 1p", "h w": "Hello h w", "p+q": "Hello p+q", "***": "Hello ***", "-ddd": "Hello -ddd" }
```

DevTools 输出时省略了引号，是输出优化，不代表它是合法标识符。

对象

- 如果键名不符合标识名的条件（比如第一个字符为数字，或者含有空格或运算符），且也不是数字，则必须加上引号，否则会报错

```
>> var obj = {  
    1p: 'Hello World'  
};
```

⚠ SyntaxError: identifier starts immediately after numeric literal [\[详细了解\]](#)

```
>> var obj = {  
    "1p": 'Hello World'  
};
```

```
← undefined
```

```
>> obj[1p]
```

⚠ SyntaxError: identifier starts immediately after numeric literal [\[详细了解\]](#)

```
>> obj["1p"]
```

```
← "Hello World"
```

对象

- 对象的键名又称为“**属性**”（property），对象的“键值”可以是任何数据类型
- 如果一个属性的值为函数，通常把这个属性称为“**方法**”，它可以像函数那样调用

```
var obj = {  
  p: function (x) {  
    return 2 * x;  
  }  
};  
  
obj.p(1) // 2
```

对象

- 如果属性的值还是对象，就形成了链式引用

```
>> var o1 = {};  
    var o2 = { bar: 'hello' };  
    o1.foo = o2;  
  
← ▶ Object { bar: "hello" }  
  
>> o1.foo.bar  
  
← "hello"  
  
>> o1.foo  
  
← ▶ Object { bar: "hello" }
```

对象o1的属性foo指向对象o2，就可以链式引用o2的属性

对象

- 对象的属性之间用逗号分隔，最后一个属性后面可以加逗号，也可以不加

```
var obj = {  
  p: 123,  
  m: function () { ... },  
}
```

m属性后面的那个逗号，有没有都可以

- 属性可以动态创建，不必在对象声明时就指定

```
var obj = {};  
obj.foo = 123;  
obj.foo // 123
```

直接对obj对象的foo属性赋值，
在运行时创建了foo属性

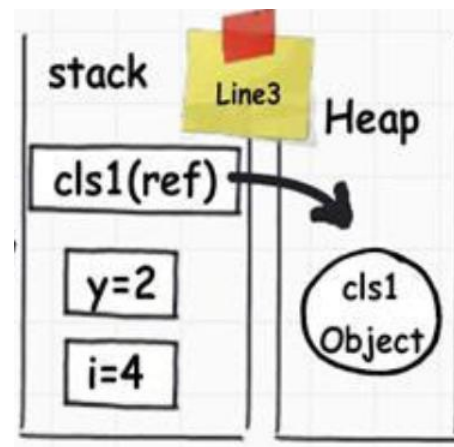
对象

- 对象的引用

- 如果不同的变量名指向同一个对象，那么它们都是这个对象的引用，也就是说指向同一个内存地址
- 修改其中一个变量，会影响到其他所有变量

```
>> var o1 = {};  
    var o2 = o1;  
← undefined  
>> o1.a = 1;  
← 1  
>> o2.a  
← 1  
>> o2.b = 2;  
← 2  
>> o1.b  
← 2
```

o1和o2指向同一个对象，为其中任何一个变量添加属性，另一个变量都可以读写该属性。



对象

- 对象的引用

— 如果取消某一个变量对于原对象的引用，不会影响到另一个变量对原对象的引用

```
>> var o1 = {"a":1,"b":2};  
    var o2 = o1;  
← undefined  
>> o2  
← ► Object { a: 1, b: 2 }  
>> o1=1;  
← 1  
>> o2  
← ► Object { a: 1, b: 2 }
```

o1和o2指向同一个对象，然后o1的值变为1，这时不会对o2产生影响，o2还是指向原来的那个对象

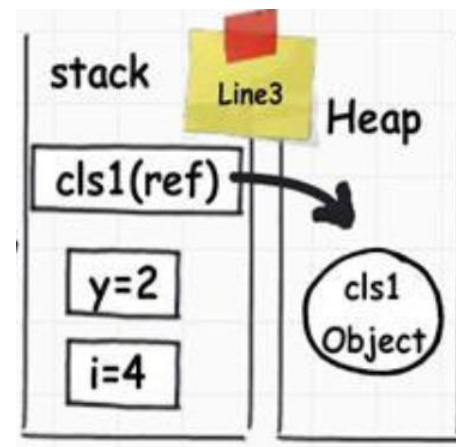
对象

- 对象的引用

- 引用只局限于对象，如果两个变量指向同一个原子类型（数字、布尔、字符串）的值，变量这时都是值的拷贝

```
>> var x = 1;  
    var y = x;  
← undefined  
  
>> x=2;  
← 2  
  
>> y  
← 1
```

当x的值发生变化后，y的值并不变，这就表示y和x并不是指向同一个内存地址



对象

- 属性的读取

- 读取对象的属性，有两种方法，一种是使用点运算符，还有一种是使用方括号运算符
- 注意，如果使用方括号运算符，键名必须放在引号里面，否则会被当作变量处理

```
>> var obj = {  
    p: 'Hello World'  
};  
  
← undefined  
  
>> obj.p  
← "Hello World"  
  
>> obj["p"]  
← "Hello World"
```

```
>> var foo = 'bar';  
← undefined  
  
>> var obj = {  
    foo:1,  
    bar:2  
};  
← undefined  
  
>> obj.foo  
← 1  
  
>> obj[foo]  
← 2  
  
>> obj['foo']  
← 1
```

引用对象obj的foo属性时，如果使用点运算符，foo就是字符串

```
>> obj.foo == obj['foo']  
← true
```

如果使用方括号运算符，但是不使用引号，那么foo就是一个变量，指向字符串bar

```
>> obj[foo] == obj['bar']  
← true
```



当心坑洞

对象

- 属性的读取

```
>> var obj = {  
  helloworld: "hello world",  
  6: "6666",  
  0.7: "77777777777"}  
← undefined  
  
>> obj['hello'+ 'world']  
← "hello world"  
  
>> obj[2+4]  
← "6666"  
  
>> obj[0.7]  
← "77777777777"  
  
>> obj[0.7] == obj['0.7']  
← true  
  
>> obj.6  
⚠ SyntaxError: unexpected token: numeric literal [详细了解]  
  
>> obj[6]  
← "6666"
```

方括号运算符内部可以使用表达式

```
>> obj['hello'+ 'world'] == obj['helloworld']  
← true  
  
>> obj[2+4] == obj[6]  
← true
```

对象

- 属性的读取

```
>> var obj = {  
  helloworld: "hello world",  
  6: "6666",  
  0.7: "77777777777"}  
← undefined  
>> obj['hello'+ 'world']  
← "hello world"  
>> obj[2+4]  
← "6666"  
>> obj[0.7]  
← "77777777777"  
>> obj[0.7] == obj['0.7']  
← true  
>> obj.6  
⚠ SyntaxError: unexpected token: numeric literal [详细了解]  
>> obj[6]  
← "6666"
```

数字键名可以不加引号，会自动转成字符串

```
>> obj[0.7]  
← "77777777777"  
>> obj[0.7] == obj['0.7']  
← true
```

对象

- 属性的读取

```
>> var obj = {  
  helloworld: "hello world",  
  6: "6666",  
  0.7: "777777777777"}  
← undefined  
>> obj['hello'+ 'world']  
← "hello world"  
>> obj[2+4]  
← "6666"  
>> obj[0.7]  
← "777777777777"  
>> obj[0.7] == obj['0.7']  
← true
```

```
>> obj.6  
⚠ SyntaxError: unexpected token: numeric literal [详细了解]  
>> obj[6]  
← "6666"
```

数值键名不能使用点运算符（因为会被当成小数点），只能使用方括号运算符

```
>> obj.6
```

⚠ SyntaxError: unexpected token: numeric literal [详细了解]

```
>> obj[6]
```

```
← "6666"
```

第一个表达式，对数值键名6使用点运算符，结果报错。第二个表达式使用方括号运算符，结果就是正确的

对象

- 属性的赋值

- 点运算符和方括号运算符，不仅可以用来读取值，还可以用来赋值
- JavaScript 允许属性的“后绑定”，也就是说，可以在任意时刻新增属性，没必要在定义对象的时候，就定义好属性

```
>> var obj = {};  
< undefined  
>> obj.foo = 'Hello';  
< "Hello"  
>> obj['bar'] = 'World';  
< "World"  
>> obj  
< ► Object { foo: "Hello", bar: "World" }
```

后绑定机制体现
JavaScript的动态特性

对象

- 属性的查看

- 使用`Object.keys`方法查看一个对象的所有属性
- 使用`Object.values`方法查看一个对象的所有属性值

```
>> var obj = {};  
← undefined  
>> obj.foo = 'Hello';  
← "Hello"  
>> obj['bar'] = 'World';  
← "World"  
>> obj  
← ▶ Object { foo: "Hello", bar: "World" }  
>> Object.keys(obj)  
← ▶ Array [ "foo", "bar" ]  
>> Object.values(obj)  
← ▶ Array [ "Hello", "World" ]
```

对象

- 属性的删除：delete 命令
 - delete 命令用于删除对象的属性，删除成功后返回true
 - 删除一个不存在的属性，delete 不报错，而且返回true，不能根据delete命令的结果判定某个属性是否存在

```
>> var obj = { p: 1 };  
< undefined  
>> Object.keys(obj)  
< ▶ Array [ "p" ]
```

```
>> delete obj.p  
< true  
>> obj.p  
< undefined
```

```
>> Object.keys(obj)  
< ▶ Array []
```

```
>> obj  
< ▶ Object { }  
>> delete obj.p  
< true
```



当心坑洞

对象

- 属性的删除：delete 命令

- delete命令只能删除对象本身的属性，无法删除继承的属性

```
>> var obj = {};  
← undefined  
>> delete obj.toString  
← true  
>> obj.toString  
← ▶ function toString()
```

toString是对象obj继承的属性，虽然delete命令返回true，但该属性并没有被删除，依然存在

这个例子还说明，即使delete返回true，该属性依然可能读取到值

详细细节参见《面向对象编程》部分



当心坑洞

对象

- 属性是否存在：in 运算符

– in运算符用于检查对象是否包含某个属性（注意，检查的是键名，不是键值），如果包含就返回true，否则返回false。它的左边是一个字符串，表示属性名，右边是一个对象

```
>> var obj = {  
    p: 'aaaaaaaaaaaaa',  
    123: 'bbbbbbbbbbbbbb'};  
← undefined  
>> p in obj  
▲ ReferenceError: p is not defined [详细了解]  
>> 'p' in obj  
← true  
>> '123' in obj  
← true  
>> 123 in obj  
← true
```

注意键名的写法
编译器无法解释

对象

- 属性是否存在：in 运算符

- in运算符不能识别哪些属性是对象自身的，哪些属性是继承的，可以使用对象的hasOwnProperty方法判断是否为对象自身的属性

```
>> var obj = {  
    p: 'aaaaaaaaaaaaa',  
    123: 'bbbbbbbbbbbbbb'};  
  
← undefined  
  
>> 'toString' in obj  
← true  
  
>> obj.hasOwnProperty('toString')  
← false
```

对象obj本身并没有toString属性，但是in运算符会返回true，因为这个属性是继承的，但不是obj自身的属性，所以hasOwnProperty方法判断false

对象

- 属性的遍历：for...in 循环
 - for...in 循环用来遍历一个对象的全部属性

```
>> var obj = {a: 1, b: 2, c: 3};  
← undefined  
>> for (var i in obj) {  
    console.log('键名: ', i);  
    console.log('键值: ', obj[i]);  
}
```

键名: a
键值: 1
键名: b
键值: 2
键名: c
键值: 3

在这个结构中，i 是一个变量名，用于代表每次迭代中的当前属性名。object 则是要遍历的对象。在循环的每次迭代中，i 会被赋值为 object 的一个属性名。

对象

- 属性的遍历：for...in 循环

- 它遍历的是对象所有可遍历（enumerable）的属性，会跳过不可遍历的属性
- 它不仅遍历对象自身的属性，还遍历继承的属性

```
>> var obj = {a: 1, b: 2, c: 3};  
← undefined  
>> for (var i in obj) {  
    console.log('键名: ', i);  
    console.log('键值: ', obj[i]);  
}  
键名: a  
键值: 1  
键名: b  
键值: 2  
键名: c  
键值: 3  
← undefined  
>> 'toString' in obj  
← true
```

对象obj继承了toString属性，该属性不会被for...in循环遍历到，因为它默认是“不可遍历”的，如果继承的属性是可遍历的，那么就会被for...in循环遍历到

Object.prototype.hasOwnProperty

Array.prototype.push

Array.prototype.pop

Array.prototype.map

不可遍历属性举例

对象

- 属性的遍历：for...in 循环

- 一般情况下，都是只想遍历对象自身的属性，所以使用for...in的时候，应该结合使用hasOwnProperty方法，在循环内部判断一下，某个属性是否为对象自身的属性

```
var person = { name: '老张' };

for (var key in person) {
  if (person.hasOwnProperty(key)) {
    console.log(key);
  }
}
// name
```

函数 - 从一个实例开始

```
<script type="text/javascript" src="js/timecountdown.js"></script>
<script type="text/javascript">
/**
 * time.day 和 time.dayZero 区别
 * time.day 如果是1天 返回 1
 * time.dayZero 如果是1天 则返回 01
 * 除了day拥有Zero外 hour,minute,second,msec都有Zero 即小于10的, 都会在前面补0
 */
window.onload=function(){
  xcsoft.countdown('2028-11-11',function (time) {
    //time.days=总天数
    document.getElementById("y1").innerHTML=time.year
    document.getElementById("d1").innerHTML=time.day
    document.getElementById("h1").innerHTML=time.hourZero
    document.getElementById("i1").innerHTML=time.minuteZero
    document.getElementById("s1").innerHTML=time.secondZero
    document.getElementById("m1").innerHTML=time.msecZero
  },function (time) {
    //倒计时结束后的操作
  })
  xcsoft.countup('2018-11-11',function (time) {
    document.getElementById("y2").innerHTML=time.year
    document.getElementById("d2").innerHTML=time.day
    document.getElementById("h2").innerHTML=time.hourZero
    document.getElementById("i2").innerHTML=time.minuteZero
    document.getElementById("s2").innerHTML=time.secondZero
  })
  var nes=parseInt(new Date().getTime()/1000)+60;
  xcsoft.countdown(parseInt(nes)+'.3',function (time) {
    document.getElementById("s3").innerHTML=time.secondZero
    document.getElementById("m3").innerHTML=time.msecZero
  })
}
</script>
```

timecountdown.js

```
xcsoft.countdown = function(a, b, c) {
  var d = new Object();
  var e = 0;
  if (typeof a == "object") {
    var f = parseInt(new Date().getTime() / 1000);
    var g = a.startTime ? parseInt(a.startTime) : 0;
    g = g == 0 ? f : g;
    var h = a.endTime;
    var x = g - f;
    d.decimal = parseInt(a.msec ? a.msec : 0)
  } else {
    var f = 0;
    var x = 0;
    d.decimal = 0;
    var h = a;
    var i = h.toString();
    if (i.indexOf('.') > 0) {
      d.decimal = i.split('.')[1];
      if (d.decimal > 3) {
        d.decimal = 3
      }
    }
  }
  d.time = h;
  d.finish = false;
  if (isNaN(h)) {
    var j = h.substring(0, 19);
    j = j.replace(/-/g, '/');
    d.time = new Date(j).getTime() / 1000
  }
}
```

函数是一段可以反复调用的代码块。函数还能接受输入的参数，不同的参数会返回不同的值。

问题：请仔细观察这些js代码，找找函数在哪里？

函数的三种声明方式

- 利用 **function** 命令声明函数
 - **function** 命令后面是函数名，函数名后面是一对圆括号，里面是传入函数的参数，函数体放在大括号里面，这种叫做**函数声明 (Function Declaration)**

```
>> function con_print(s) {  
    console.log(s);  
}  
← undefined  
>> con_print("hello world");  
    hello world  
← undefined
```

函数的三种声明方式

- 函数表达式

- 采用变量赋值的写法将一个函数赋值给变量，因为赋值语句的等号右侧只能放表达式，所以这个函数又称**函数表达式 (Function Expression)**

```
>> var print = function(s) {  
    console.log(s);  
};  
  
← undefined  
  
>> print  
← ► function ()  
  
>> print("hello world")  
← "hello world"
```

函数的三种声明方式

- 函数表达式

— 采用函数表达式声明函数时，**function**命令后面不带有函数名时为匿名函数。如果加上函数名，该函数名只在函数体内部有效，在函数体外部无效

```
>> var print1 = function x(){  
    console.log(typeof x);  
};  
← undefined  
>> x  
▲ ReferenceError: x is not defined [详细了解]  
>> print1    这是对print1变量的引用  
← ▶ function x()  
>> print1()  这是对print1函数的调用  
    function  
← undefined
```

```
>> var fib = function f(num) {  
    if (num === 0) return 0;  
    if (num === 1) return 1;  
    return f(num - 2) + f(num - 1);  
};  
← undefined  
>> fib(6)  
← 8
```

上面代码在函数表达式中，加入了函数名x。这个x只在函数体内部可用，指代函数表达式本身，其他地方都不可用。这种写法的用处有两个，一是可以在函数体内部调用自身，二是方便除错（除错工具显示函数调用栈时，将显示函数名，而不再显示这里是一个匿名函数）

函数的三种声明方式

- 函数的表达式需要在语句的结尾加上分号，表示语句结束
- 而函数的声明在结尾的大括号后面不用加分号
- 总的来说，这两种声明函数的方式，差别很细微，可以近似认为是等价的

函数的三种声明方式

- **Function 构造函数**

- 可以传递任意数量的参数给Function构造函数，只有最后一个参数会被当做函数体，如果只有一个参数，该参数就是函数体。可以不使用new命令，返回结果完全一样。不直观，很少用

```
var add = new Function(  
    'x', 形参名  
    'y',  
    'return x + y'  
);      函数体  
  
// 等同于  
function add(x, y) {  
    return x + y;  
}
```

```
var foo = new Function(  
    'return "hello world";'  
);  
  
// 等同于  
function foo() {  
    return 'hello world';  
}
```

函数的重复声明

- 如果同一个函数被多次声明，**后面的声明就会覆盖前面的声明**。由于**函数名提升现象**，前一次声明在任何时候都是无效的

```
>> function f() {  
    console.log(1);  
}  
← undefined  
>> function f() {  
    console.log(2);  
}  
← undefined  
>> f()  
2
```

回顾：变量提升



当心坑洞

函数的调用

- 调用函数时，要使用圆括号运算符，圆括号之中可以加入函数的参数
- 函数体内部的return语句表示返回。JavaScript引擎遇到return语句，就直接返回return后面的那个表达式的值，后面即使还有语句也不会得到执行
- return语句不是必需的，如果没有的话，该函数就不返回任何值，或者说返回undefined
- 函数可以调用自身，这就是递归（recursion）

函数的调用

函数调用，传参与返回值

```
function add(x, y) {  
    return x + y;  
}  
  
add(1, 1) // 2
```

通过递归，计算斐波那契数列

```
function fib(num) {  
    if (num === 0) return 0;  
    if (num === 1) return 1;  
    return fib(num - 2) + fib(num - 1);  
}  
  
fib(6) // 8
```

函数是第一等公民

- JavaScript 语言将函数看作一种值，与其它值（数值、字符串、布尔值等等）地位相同
- 凡是可以使用值的地方，就能使用函数。可以把函数赋值给变量和对象的属性，也可以当作参数传入其他函数，或者作为函数的结果返回。函数只是一个可以执行的值，并无特殊之处
- 由于函数与其他数据类型地位平等，所以在 JavaScript 语言中又称**函数为第一等公民**

函数是第一等公民

```
>> function add(x, y) {  
    return x + y;  
}
```

```
← undefined
```

```
>> var operator = add;
```

```
← undefined
```

```
>> operator
```

```
← ► function add()
```

```
>> function a(op){  
    return op;  
}
```

```
← undefined
```

```
>> a(add)(1,1)
```

```
← 2
```

```
>> a(operator)(1,1)
```

```
← 2
```

可以把函数赋值给变量

可以把函数当作参数传入其他函数

函数名的提升

- JavaScript 引擎将函数名视同变量名，所以采用function命令声明函数时，整个函数会像变量声明一样，被提升到代码头部，也即所谓**变量提升 (hoisting)**

```
>> f2();  
      function f2() {}  
← undefined
```

表面上，上面代码好像在声明之前就调用了函数f。但是实际上，由于“变量提升”，函数f被提升到了代码头部，也就是在调用之前已经声明了

函数名的提升

- 如果采用赋值语句定义函数，JavaScript 就会报错

```
>> f1();  
var f1 = function (){};
```

⚠️ **TypeError: f1 is not a function** [\[详细了解\]](#)

=

```
>> var f1;  
f1();  
f1 = function(){};
```

⚠️ **TypeError: f1 is not a function** [\[详细了解\]](#)

代码第二行，调用f1的时候，f1只是被声明了，还没有被赋值，所以会报错。

- 如果同时采用function命令和赋值语句声明同一个函数，**最后总是采用赋值语句的定义**

```
>> var f3 = function () {  
  console.log('1');  
}
```

```
function f3() {  
  console.log('2');  
}
```

```
← undefined
```

```
>> f3()
```

```
1
```

变量提升

=

等价于

```
>> var f4;  
function f4() {  
  console.log('2');  
}
```

```
f4 = function () {  
  console.log('1');  
}
```

```
← ▶ function f4()
```

```
>> f4()
```

```
1
```



当心坑洞

函数的name属性

- 函数的name属性返回函数的名字

```
>> function f1() {}  
    f1.name // "f1"  
  
← "f1"  
  
>> var f2 = function () {};  
    f2.name // "f2"  
  
← "f2"  
  
>> var f3 = function myf3() {};  
    f3.name // myf3  
  
← "myf3"
```

如果是通过变量赋值定义的函数，如果变量的值是一个匿名函数，那么name属性返回变量名

如果是通过变量赋值定义的函数，如果变量的值是一个具名函数，那么name属性返回函数表达式的名字

函数的name属性

- name属性的用处之一是获取参数函数的名字

```
>> var myFunc = function () {};  
    var myFunc1 = function f1() {};  
    function func_name(f) {  
      console.log(f.name);  
    }  
← undefined  
  
>> func_name(myFunc)  
    myFunc  
← undefined  
  
>> func_name(myFunc1)  
    f1
```

函数的length属性

- 函数的length属性返回函数**预期传入**的参数个数，即函数定义之中的参数个数

```
>> function f(a,b){}  
← undefined  
>> f.length  
← 2  
>> function g(){}  
← undefined  
>> g.length  
← 0
```

函数的toString方法

- 函数的toString方法返回一个字符串，内容是函数的源码，包括函数内的注释

```
>> function func(a,b){  
    //calculate and return sum of a,b  
    return a+b;  
}  
← undefined  
  
>> func.toString()  
← "function func(a,b){  
    //calculate and return sum of a,b  
    return a+b;  
}"
```

函数作用域

- 作用域 (scope) 指的是变量存在的范围，在 ES5 的规范中，**Javascript 只有两种作用域**：
 - 一种是全局作用域，变量在整个程序中一直存在，所有地方都可以读取
 - 一种是函数作用域，变量只在函数内部存在

ES6 又新增了块级作用域，本教程不涉及

函数作用域

- 函数外部声明的变量就是**全局变量**（global variable），它可以在函数内部读取
- 在函数内部定义的变量，外部无法读取，称为**局部变量**（local variable）

```
>> var v = 1;

    function f() {
      console.log(v);
    }
← undefined
>> f()
1 全局变量
```

```
>> function f(){
      var vv = 1;
    }
← undefined
>> vv
⚠ ReferenceError: vv is not defined [详细了解]
```

局部变量

函数作用域

- 函数内部定义的变量，会在该作用域内覆盖同名全局变量

```
>> var v = 1;

function f(){
  var v = 2;
  console.log(v);
}
← undefined
>> f()
2
← undefined
>> v
1
← 1
```

- 变量v同时在函数的外部和内部有定义
- 在函数内部定义，局部变量v覆盖了全局变量v，输出为2
- 全局变量的值不受局部变量的影响，仍为1

函数作用域

- 对于var命令来说，局部变量只能在函数内部声明，在其他区块中声明，一律都是全局变量，只有函数所在的区块会造成作用域变化

```
>> if (true) {  
    var x = 5;  
}  
console.log(x); // 5  
5
```

```
>> for (i=1;i<3;i++){  
    var xx = i;  
    console.log(i);  
}  
1  
2  
← undefined  
>> xx  
← 2
```

xx是全局变量

函数内部的变量提升

- 与全局作用域一样，函数作用域内部也会产生变量提升现象，在函数内，var命令声明的变量，不管在什么位置，变量声明都会被提升到函数体的头部

```
function foo(x) {  
  if (x > 100) {  
    var tmp = x - 100;  
  }  
}
```

// 等同于

```
function foo(x) {  
  var tmp;  
  if (x > 100) {  
    tmp = x - 100;  
  };  
}
```

函数本身的作用域

- 函数本身也是一个值，也有自己的作用域
- 函数的作用域与变量一样，就是其声明时所在的作用域，与其运行时所在的作用域无关

上面代码中，函数x是在函数f的外部声明的，所以它的作用域绑定外层，内部变量a不会到函数f体内取值，所以输出1，而不是2。

```
>> var a = 1;  
    var x = function () {  
      console.log(a);  
    };  
  
    function f() {  
      var a = 2;  
      x();  
    }  
  
    < undefined  
>> f()  
    1
```

声明时作用域

运行时作用域



当心坑洞

函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

```
>> function f(a, b) {  
    return a;  
}  
  
← undefined  
>> f(1,2,3)  
← 1  
>> f(1)  
← 1  
  
>> f()  
← undefined  
>> f.length  
← 2  
>> f(,1)  
▲ SyntaxError: expected expression, got ',' [详细了解]  
>> f(undefined, 2)  
← undefined
```

函数f定义了两个参数，但是运行时无论提供多少个参数（或者不提供参数），**JavaScript 都不会报错**



函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

函数的length属性与实际传入的参数个数无关，**只反映函数预期传入的参数个数**

```
>> function f(a, b) {  
    return a;  
}  
← undefined  
>> f(1,2,3)  
← 1  
>> f(1)  
← 1  
>> f()  
← undefined  
>> f.length  
← 2  
>> f(,1)  
▲ SyntaxError: expected expression, got ',' [详细了解]  
>> f(undefined, 2)  
← undefined
```

函数的参数

- 函数参数不是必需的，JavaScript 允许省略参数，省略的参数的值就变为undefined

```
>> function f(a, b) {  
    return a;  
}  
← undefined  
>> f(1,2,3)  
← 1
```

没有办法只省略靠前的参数，而保留靠后的参数。如果一定要省略靠前的参数，只有显式传入undefined

```
← undefined
```

```
>> f.length
```

```
← 2
```

```
>> f(,1)
```

```
⚠ SyntaxError: expected expression, got ',' [详细了解]
```

```
>> f(undefined, 2)
```

```
← undefined
```

函数的参数

- 如果有同名的参数，则取最后出现的那个值，即使后面的没有值或被省略，也是以其为准

```
function f(a, a) {  
  console.log(a);  
}
```

```
f(1, 2) // 2
```

```
function f(a, a) {  
  console.log(a);  
}
```

```
f(1) // undefined
```

- 函数f有两个参数，且参数名都是a，取值的时候，以后面的a为准
- 如果调用函数f的时候没有提供第二个参数，a的取值就变成了undefined

参数的两种传递方式

- 函数参数如果是原始类型的值（数值、字符串、布尔值），传递方式是传值传递（passes by value），在函数体内修改参数值，不会影响到原始值
- 如果函数参数是复合类型的值（数组、对象、其他函数），传递方式是传址传递（pass by reference），传入函数的是原始值的地址，因此在函数内部修改参数，将会影响到原始值

参数的两种传递方式

```
var p = 2;
```

```
function f(p) {
```

```
  p = 3;
```

```
}
```

```
f(p);
```

```
p // 2
```

VS

```
var obj = { p: 1 };
```

```
function f(o) {
```

```
  o.p = 2;
```

```
}
```

```
f(obj);
```

```
obj.p // 2
```

传值

在函数内部，p的值是原始值的拷贝，
无论怎么修改，都不会影响到原始值

传地址

传入函数的原始值的地址，因此在函数内部修改参数，将会影响到原始值

参数的两种传递方式

- 如果函数内部修改的，不是参数对象的某个属性，而是把参数对象整个替换成另一个值，这时**不会影响到原始值**

```
>> var obj = [1, 2, 3];  
  
    function f(o) {  
      o = [2, 3, 4];  
    }  
    f(obj);  
← undefined  
  
>> obj  
← ► Array(3) [ 1, 2, 3 ]
```

形式参数（o）的值实际是参数obj的**地址副本**，重新对o赋值导致o指向另一个地址，保存在原地址上的值不受影响

arguments 对象

- arguments对象只在函数体内部才可以使用，包含了函数运行时的所有参数，可以在函数体内部读取所有参数，通过arguments对象的length属性，可以判断函数调用时到底带几个参数

```
>> var f = function (one) {  
    console.log(arguments.length+" arguments:");  
    console.log("arg0: "+ arguments[0]);  
    console.log("arg1: "+ arguments[1]);  
    console.log("arg2: "+ arguments[2]);  
}  
← undefined  
>> f(1,2,3,4,5)  
5 arguments:  
arg0: 1  
arg1: 2  
arg2: 3  
← undefined  
>> f(1,2)  
2 arguments:  
arg0: 1  
arg1: 2  
arg2: undefined  
← undefined
```

arguments 对象

- 正常模式下，arguments对象可以在运行时修改

```
var f = function(a, b) {  
  arguments[0] = 3;  
  arguments[1] = 2;  
  return a + b;  
}  
  
f(1, 1) // 5
```

- **严格模式**下arguments对象是一个只读对象，修改它是无效的，但不会报错

```
var f = function(a, b) {  
  'use strict'; // 开启严格模式  
  arguments[0] = 3; // 无效  
  arguments[1] = 2; // 无效  
  return a + b;  
}  
  
f(1, 1) // 2
```

在JavaScript中使用"use strict"指令时，会启用严格模式。严格模式是一种更加严格的JavaScript解析和错误处理方式，它有助于减少一些常见的错误，并使代码更具健壮性。

1. 禁止使用八进制数字字面量（如0123）。
2. 禁止使用with语句。

变量必须先声明后使用。未声明的变量将会抛出错误。

闭包 (Closures)

- 闭包 (closure) 是 JavaScript 语言的一个难点，也是它的特色，很多高级应用都要依靠闭包实现
- 闭包是函数和声明该函数的词法环境的组合

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
<script>
  var a = 10;
  function fa(){
    a--;
    console.log(a);
  }
</script>
```

采用全局变量存在的问题：

- 1、占用内存
- 2、存在变量污染问题

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
function fa(){  
  let a = 10;  
  a--;  
  console.log(a);  
}
```

let 声明的变量的作用域则是它当前所处代码块，即它的作用域既可以是全局或者整个函数块，也可以是 if、while、switch 等用 {} 限定的代码区块

这样就可以了。。。

吗？

每次函数一调用完，就会销毁，a 永远都是 10 开始，存不住

闭包 (Closures)

- 问题提出：做一个函数，这个函数每执行一次就累积减一

```
function fa(){  
  let a = 10;  
  function fb(){  
    a--;  
    console.log(a);  
  }  
  return fb;  
}  
var fm = fa();
```

```
> fm();  
9  
< undefined  
> fm();  
8  
< undefined  
> fm();  
7  
< undefined  
>
```

- 1、有函数嵌套
- 2、内部函数引用外部作用域的变量参数
- 3、返回值是函数
- 4、创建一个对象函数，让其长期驻留

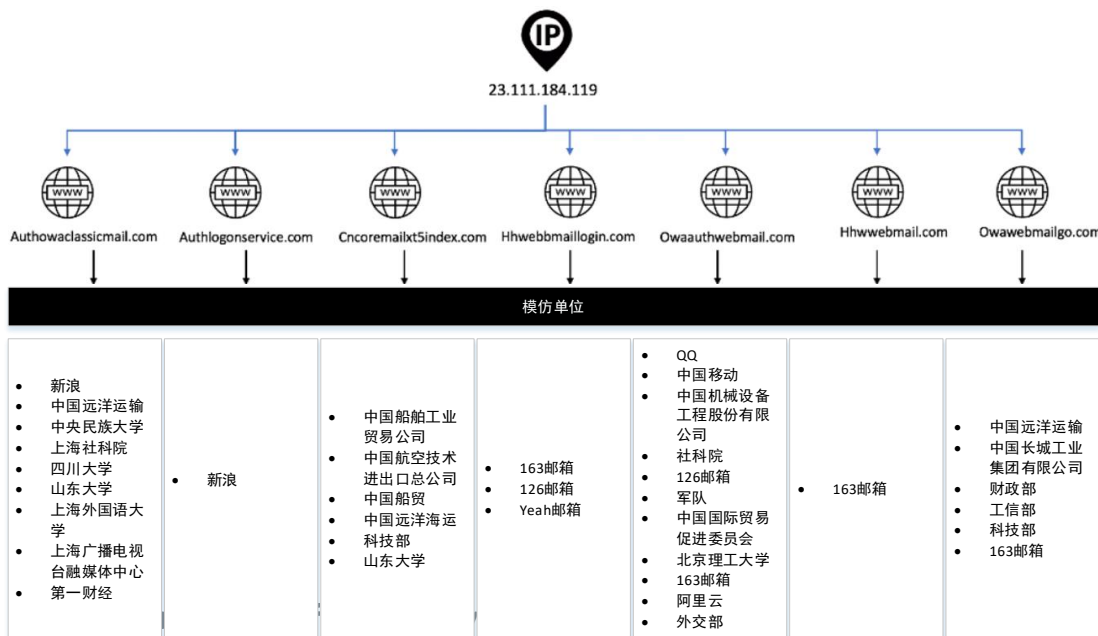
fa() 执行结束，按理 a 应该被销毁
但因为 fb 还在用 a，所以
JavaScript 不会销毁 a
a 被“闭合”在 fb 的词法作用域里——这就是闭包的本质

我们需要一种机制，即能长期保存变量又不污染全局，这就是闭包

即使 fa 已经执行完毕，a 却继续存在、继续减少、继续被访问。

<https://juejin.cn/post/7084549768067678245>
School of Cyber Security, BUPT #80

网安小知识—印度对我国的钓鱼攻击



Statistics

Systems

Tasks

Log

Logout

SNo	IP	Computer	User	Operating system	First Seen	Last Seen
1	106.111.111.5	AS-PC	Ash-PC	Windows 10 Pro	2019-09-23	2019-09-23 11:03:39
2	121.61.111.10	MC-934	mo-934	Windows 7 Ultimate	2019-09-26	2019-10-11 08:53:34
3	58.24.111.14	DE-BSPD9L	DE-SPDD9L	Windows 10 Education	2019-09-27	2019-10-24 08:15:44
4	66.24.111.7	WIF-POUVSUP	WIN-POUVSUP	Windows Server 2016 Standard	2019-10-04	2019-10-23 06:28:40
5	91.20.111.0	AC-S-PC	Ac-PC	Windows 7 Professional	2019-10-05	2019-10-05 10:31:02
6	223.7.111.31	W-PC	w-PC	Windows 10 Pro	2019-10-09	2019-10-21 03:48:06
7	45.111.111.20	L-E9JIPJQ	L-E9JIPJQ	Windows 10 Home China	2019-10-10	2019-10-10 10:43:04
8	185.2.111.173	A-PC	A-PC	Windows 7 Professional	2019-10-16	2019-10-16 02:34:44
9	114.111.111.0.4	H-PC	h-PC	Windows 7 Professional	2019-10-16	2019-10-21 07:52:23
10	65.111.111.100	C-OWSTHIN	C-JowsThin	Windows Embedded Standard	2019-10-17	2019-10-23 02:01:43
11	17.111.111.7.100	DI-PC	E-PC	Windows%207x20Enterprise	2019-10-21	2019-10-21 06:42:25
12	61.111.111.204	HK-34	F-234	Microsoft Windows XP	2019-10-23	2019-10-23 02:43:48

Submit Task

Select

Go